01 / MOTION PATHWAY

How our model extracts motion

A mathematical walkthrough from pixel values to local evidence, prediction and remembered change

The core question is simple: **which small shift makes an earlier image patch line up with a later one?** The model asks this at 100 locations, over three temporal separations. It keeps a score for every candidate shift, allowing later computations to use both the preferred motion and the alternatives.

This document follows the actual backbone used by the 19-task suite. The source was checked against the running continuation07 source hashes on October 12, 2026. Equations reproduce that implementation; the numerical demonstrations use generated inputs on CPU. No training settings or checkpoints were changed.

Operation	What it contributes
Pixels and overlapping crops	A local light-dark pattern at each image location.
Mean subtraction and windowing	Emphasis on local structure and the crop interior.
81 normalized shift scores	Evidence about correspondence between two observations.
Learned refinement and softmax	A distribution over candidate displacements.
Three lags and a weighted mean	Short- and longer-interval motion, plus a compact velocity estimate.
Prediction and temporal comparison	Whether motion continues and how local evidence changes.
Shared spatial attention	Which location contributes to memory and the response.

How to read the equations

Every symbol is introduced where it first appears. A scalar is one number; a vector is an ordered list; a matrix is a table of numbers. A sum adds entries, an average divides their sum by their count, and a norm measures the size of a vector. Pages 2-9 explain the motion measurement itself. Pages 10-15 explain how it becomes useful to the task. Pages 16-20 provide checked examples, theory, a glossary and sources.

Implementation anchors: SecondPass/StabilizedAttentionKrauzlis/model.py and SecondPass/MemoryGuidedSearch/train.py. The architecture and training-source hashes are recorded with the accompanying verification receipt.

1. Start with the actual input

Intensity is a pixel value; movement is a relationship between pixel patterns at different times.

A pixel is one addressable cell of an image. **Grayscale intensity** is its encoded light-dark value. In this input convention, 0 is black, 1 is white and 0.5 is midpoint gray. These numbers are dimensionless. A color pixel contains three component values: red, green and blue, abbreviated RGB. The motion branch takes their arithmetic mean.

$$I_t(x, y) = \frac{X_{t,R}(x, y) + X_{t,G}(x, y) + X_{t,B}(x, y)}{3}$$

X denotes the input color image; R, G and B identify its channels. I denotes the resulting grayscale image. The subscript t selects a frame. Coordinates x and y select a column and row, with x increasing rightward and y downward. Each runs from 0 to 99. Thus $I_5(20,30)=0.8$ means one particular pixel in frame 5 has grayscale value 0.8.

A numerical example

At one pixel, RGB = (0.9, 0.6, 0.3) becomes I = 0.6. The arithmetic conversion is shared at every pixel. Equal-channel gray (0.6, 0.6, 0.6) produces the same motion input. Color information also travels through the separate appearance branch.

$$\mathbf{d} = (x_2 - x_1,\ y_2 - y_1), \qquad \mathbf{v} = \mathbf{d}/\Delta t$$

The bold symbol d is displacement: two numbers describing a position change. If a dot moves from (20,30) to (24,28), d=(4,-2) pixels. The symbol Δt is the elapsed interval, and v is displacement divided by that interval. With two frame steps between the observations, v=(2,-1) pixels per frame step. A negative vertical component means upward movement.

Why a single image is insufficient

A bright dot at (20,30) does not by itself tell us where it came from. A second observation supplies another pattern, and matching the patterns supplies displacement evidence. The computation operates on image arrays; its frame steps acquire physical durations only when a presentation rate is specified.

Source: model.py, forward(), lines 88-96. This document uses t as an integer frame index throughout.

2. Where the model looks

A site is one position on the 10 by 10 analysis grid.

There are 100 sites: 10 rows and 10 columns. Each site has a 24 by 24 grayscale crop containing 576 values. Crops start 10 pixels apart, so nearby sites see some of the same pixels. That shared input is what **overlapping evidence** means.

$$x = 10c - 7 + a, \qquad y = 10r - 7 + b, \qquad i = 10r + c$$

Here r and c are site row and column, each 0 through 9. The single index i labels the site, 0 through 99. Crop coordinates a and b each run from 0 through 23. The crop center is (10c+4.5, 10r+4.5). For site (4,4), the crop covers x=33 through 56 and y=33 through 56. Its neighboring crop at (4,5) covers x=43 through 66.

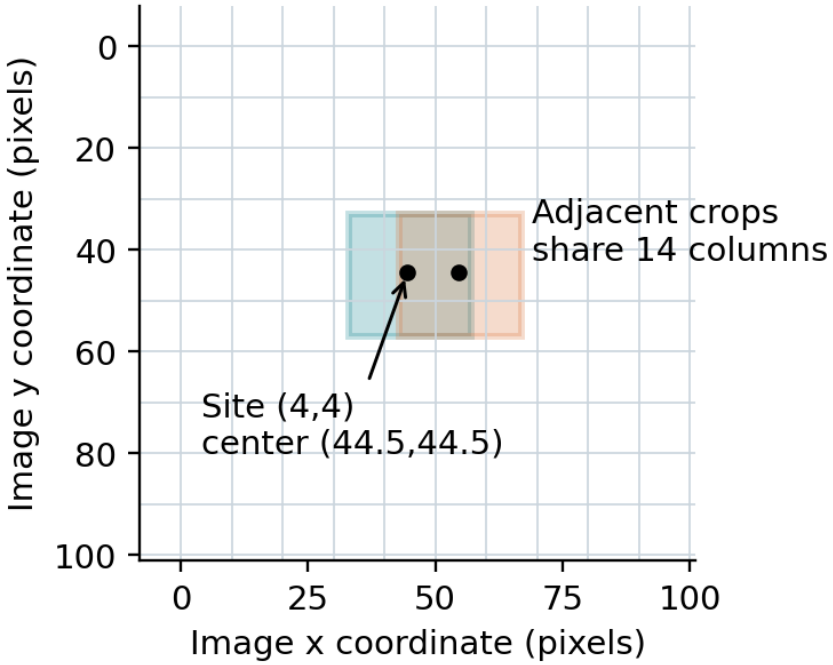


Figure 1. Exact crop geometry. The highlighted neighboring crops overlap in 14 columns: 24 minus the 10-pixel stride.

The model first subtracts 0.5 from the full grayscale image, then pads it with seven zeros on every side. A padded value therefore represents midpoint gray in the original input convention. Padding lets the same crop rule work at the outermost sites.

Why this scale? A larger local neighborhood lets moving structure remain available to matching. A smaller stride makes neighboring measurements overlap. The values 24 and 10 are fixed architectural choices: training adjusts learned weights, but it does not resize these crops or relocate the grid. The attention display assigns each site a 10 by 10 image tile; its motion measurement uses the larger crop.

3. Prepare the crop

Remove the local average and gradually reduce the influence of the boundary.

Let $Q_{t,i}(a,b)$ be the crop after the global subtraction of 0.5 and padding. Compute its mean over all 576 entries. Subtracting that mean turns brighter-than-average entries positive and darker-than-average entries negative.

$$\overline{Q}_{t,i} = \frac{1}{576} \sum_{a=0}^{23} \sum_{b=0}^{23} Q_{t,i}(a,b)$$

$$P_{t,i}(a,b) = [Q_{t,i}(a,b) - \overline{Q}_{t,i}]\, w(a) w(b)$$

$$w(n) = \frac{1}{2}\left[1 - \cos\left(\frac{2\pi n}{23}\right)\right], \quad n = 0, \ldots, 23$$

P names the prepared crop. The function w is a nonperiodic Hann window: it is zero at the endpoints and close to one near the center. Multiplying a horizontal weight by a vertical weight gives the two-dimensional weight at each crop pixel. The symbol π is the circle constant; cosine gives the smooth taper.

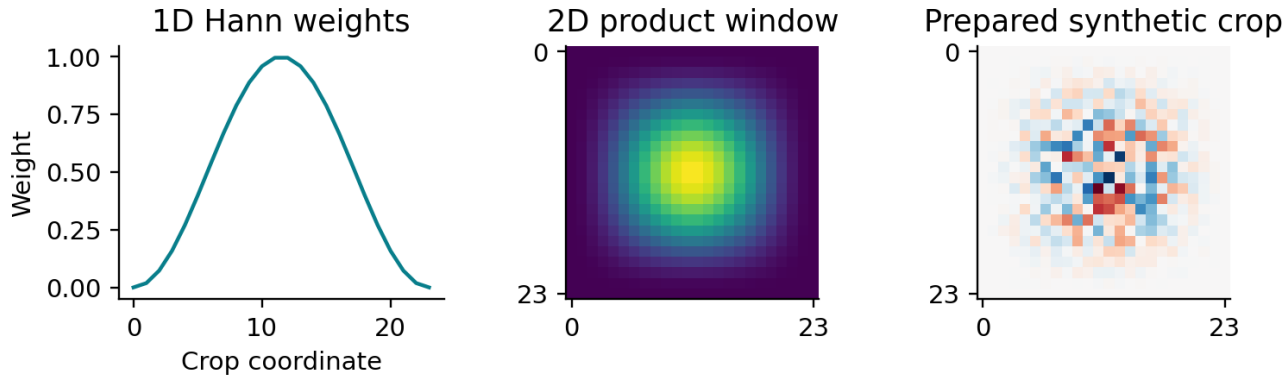


Figure 2. Window weights and a prepared synthetic crop. Red/blue in the last panel indicate positive/negative deviations about the crop mean.

Why subtract the mean? A uniform brightness level would contribute a broad positive match at many shifts. Centering emphasizes variation within the crop. Adding the same constant everywhere in an interior crop leaves the prepared pattern unchanged. At image borders, padding also participates in the mean.

Why taper? A sharply cut edge of a crop can act like an artificial image edge. Windowing emphasizes central structure and makes boundary values fade. Both observations receive a window fixed to the crop coordinates. A moving feature therefore receives slightly different weights as it moves; even a correct match can score below one.

Subtraction and tapering are distinct operations. The crop has zero mean immediately after subtraction; multiplication by the nonuniform window can change that mean again.

4. Ask 81 correspondence questions

For each candidate displacement, multiply aligned values and add the products.

Choose one site and two prepared crops. Call the earlier crop A and the current crop B. A candidate displacement $d=(d_x,d_y)$ says that structure at (a,b) in A should appear at $(a+d_x,b+d_y)$ in B. Each component can be any integer from -4 through +4, giving 9 times 9 = 81 alternatives.

$$N(\mathbf{d}) = \sum_{a,\,b} A(a,b)\,B(a+d_x, b+d_y)$$

N is the unnormalized correlation score. The summation adds products over crop coordinates. For this equation, treat values beyond the 24 by 24 crop as zero. A positive value meeting a positive value contributes positively; two negative values also contribute positively. Opposite signs reduce the score. A displacement that aligns the light-dark structure tends to produce a large sum.

Work through a tiny one-dimensional example

Take an already prepared row A=[0,1,-1,0,0] and a later row B=[0,0,1,-1,0]. At displacement 0, the nonzero product is (-1) times 1, giving N(0)=-1. At displacement +1, the aligned products are 1 times 1 and (-1) times (-1), giving N(+1)=2. The larger score supports a one-pixel shift to the right. The two-dimensional operation performs the same arithmetic in both axes.

$$\mathcal{D} = \{(d_x, d_y) : d_x, d_y \in \{-4, -3, \ldots, 4\}\}$$

The symbol D in this equation is the set of candidate shifts. Arranging the scores as a 9 by 9 table gives a **displacement map**. Its rows indicate vertical shifts and its columns horizontal shifts. It is a map of motion alternatives for one image location. Repeating the calculation at all 100 sites gives 100 such tables for each time lag.

Why keep all scores? A repeating pattern may align at several shifts. A long straight edge can constrain motion across the edge while leaving motion along it ambiguous. Keeping the whole table preserves these alternatives for later computation. Correlation volumes also appear in learned optical-flow systems such as FlowNet [1]. Our score volume is built directly from prepared grayscale crops.

5. Separate shape from signal strength

Normalization measures alignment; a separate factor records how much prepared signal is present.

$$E_A = \sum_{a,b} A(a,b)^2, \quad E_B = \sum_{a,b} B(a,b)^2, \quad z = \sqrt{E_A E_B}$$

$$C(\mathbf{d}) = \frac{N(\mathbf{d})}{\max(z, 10^{-8})}, \qquad r = \frac{z}{z + 0.01}$$

E_A and E_B are sums of squared prepared pixel values, called crop energies. Squaring makes both positive and negative deviations contribute. z is the geometric mean of the two energies. C is the normalized correspondence score. The maximum in its denominator prevents division by a value smaller than 10^{-8}. The scalar r, between zero and one, is the code's energy-based reliability factor.

In the small row example, both energies equal 2, so z=2. The correct-shift score becomes 2/2=1 and the zero-shift score becomes -1/2=-0.5. If A is multiplied by a positive factor a and B by a positive factor b, the product sum and z both scale by ab. Their ratio stays the same when the numerical floor is inactive.

z	r = z / (z + 0.01)	Interpretation
0	0	No prepared variation in at least one crop.
0.01	0.5	The energy factor gives half weight.
0.09	0.9	Substantial prepared signal.
0.99	0.99	The factor is close to its upper limit.

What exactly is normalized? The denominator uses the energies of the two full prepared crops and is shared across all candidate shifts. The code does not recompute a different denominator or a different local mean for each overlap region. This precise rule matters when reproducing the score.

What does reliability mean here? It measures available prepared-image energy. Rich random noise can have high energy, and a periodic texture can have high energy but several plausible shifts. The displacement table carries match structure; r supplies a separate signal-strength measure. At flat midpoint gray, both C and r are exactly zero.

Code names: costs = C; valid = r. These are similarity scores, so larger means a better match. The name valid denotes energy weighting, not an explicit check that a full temporal lag has elapsed.

6. The FFT computes the same sums

The Fourier transform changes the arithmetic, while the question remains alignment under a shift.

A Fourier transform expresses an array through sinusoidal components. Each component has a magnitude and a phase; phase specifies its alignment. For a finite array the transform is an invertible change of coordinates. FFT means fast Fourier transform, an efficient algorithm for calculating this change. Multiplication of Fourier coefficients can compute all the correlation shifts together.

$$\widehat{A} = \mathcal{F}_{32}(A), \qquad \widehat{B} = \mathcal{F}_{32}(B)$$

$$N_{32} = \mathcal{F}_{32}^{-1}(\overline{\widehat{A}}\,\widehat{B})$$

The calligraphic F denotes the two-dimensional Fourier transform on a 32 by 32 array. A hat denotes its coefficients. The overbar here means complex conjugation: reverse the sign of the imaginary component. It has a different role from the crop-mean bar on page 4, because its operand is a complex coefficient. The inverse transform returns a real 32 by 32 table of shift scores [2].

Why the order matters

Conjugating the **earlier** spectrum and multiplying by the later spectrum produces the forward-shift convention on page 5. A rightward displacement peaks at positive d_x. Swapping which spectrum is conjugated reverses that convention. In the code, spectra[:,ix].conj() is the earlier term and spectra is the current term.

$$\text{index}(d_x, d_y) = 32(d_y \bmod 32) + (d_x \bmod 32)$$

The operator mod gives a remainder between 0 and 31. It maps a negative shift such as -1 to array index 31. Flattening the 32 by 32 result then uses row times 32 plus column. The model retrieves only the 81 entries for shifts -4 through +4.

Why pad 24 to 32?

A discrete FFT gives circular correlation: indices wrap at 32. Each crop occupies coordinates 0 through 23 and the remaining eight positions contain zeros. For the retained shifts of at most four pixels per axis, wrapped signal cannot meet the opposite edge of the other 24-pixel crop. Consequently these retained entries equal the finite, zero-extended product sums. Full correlation at every possible shift would require a larger array.

Independent CPU check: all 81 retained normalized FFT scores matched explicit sums; maximum absolute difference 4.5e-08. The checked code uses rfft2/irfft2 to exploit the symmetry of real-valued inputs.

7. Turn scores into a population

A small learned correction adjusts the score map; softmax makes the weights sum to one.

$$\widetilde{C} = C + 0.1\, f_\theta(C), \qquad f_\theta :\ 1 \to 4 \to 1 \text{ channels}$$

C is one 9 by 9 score map. The learned function f_θ applies a 3 by 3 convolution from one channel to four, a GELU nonlinearity, and another 3 by 3 convolution back to one. A convolution forms local weighted sums shared across map positions. Here those positions are **candidate displacements**. A neighboring cell represents a similar possible shift. GELU makes the correction depend nonlinearly on those sums.

The same correction network is used at every image site, time and lag. Both layers use padding one to preserve 9 by 9 shape. The last layer starts with zero weights and bias, so the initial correction is exactly zero. The fixed factor 0.1 scales its contribution. The learned parameters θ can reshape nearby peaks as task learning proceeds.

$$p_k = \frac{\exp(\widetilde{C}_k/\tau_\ell)}{\sum_{j=1}^{81} \exp(\widetilde{C}_j/\tau_\ell)}, \qquad \sum_{k=1}^{81} p_k = 1$$

k indexes one candidate shift; j indexes all candidates in the denominator. p_k is its normalized weight. The positive number τ_ℓ, called temperature, is separate for each lag ℓ. It starts at 0.04 and is computed as exp of a learned parameter, clamped to [0.015, 0.2]. A smaller temperature magnifies score differences before normalization.

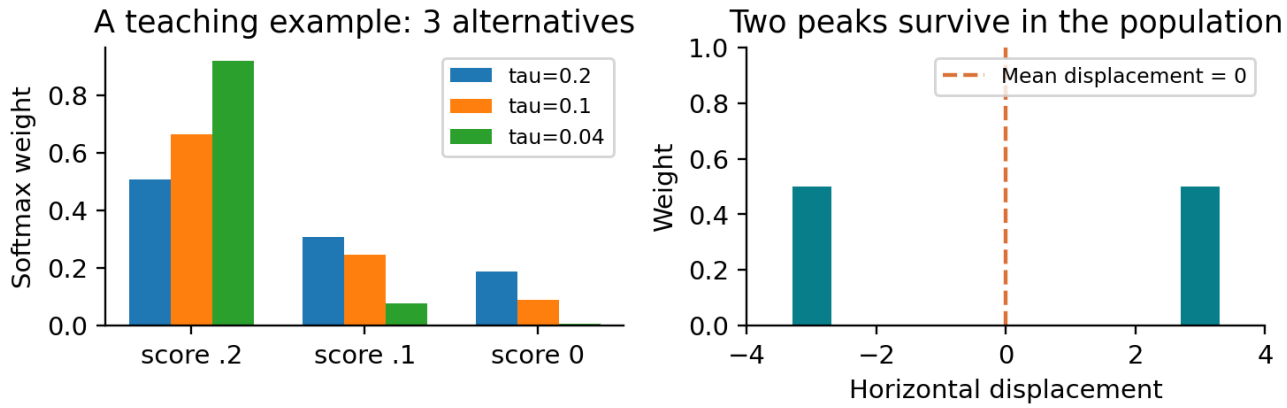


Figure 3. Left: a three-alternative teaching example. Right: a distribution can express two opposing shifts even when its mean is zero.

The odds ratio is $\exp[(\tilde{C}_i - \tilde{C}_j)/\tau]$. A score advantage of 0.1 at τ=0.04 gives about 12.2 times as much weight. The term **population** means the entire 81-number vector, with each coordinate assigned a candidate displacement. These normalized weights can be used as a discrete distribution in subsequent arithmetic.

8. Three lags, one velocity convention

Displacement accumulated over four frames must be divided by four to obtain pixels per frame.

$$s(t, \ell) = \max(t - \ell, 0), \qquad \ell \in \{1, 2, 4\}$$

s is the earlier frame index. The model compares current frame t to frames one, two and four steps earlier. When the requested earlier index would be negative, it uses frame zero. Each lag has its own 81-entry population $p_{t,\ell,i}$. The index i identifies the spatial site.

$$\widehat{\mathbf{d}}_{t,\ell,i} = \sum_{k=1}^{81} p_{t,\ell,i,k}\, \mathbf{d}_k, \qquad \widehat{\mathbf{v}}_{t,\ell,i} = \widehat{\mathbf{d}}_{t,\ell,i}/\ell$$

The hat marks an estimate. d_k is the fixed two-component displacement for candidate k. Multiply each candidate vector by its weight and add: this is the expected displacement. Dividing by the nominal lag gives velocity. For steady rightward motion of one pixel per frame, the expected displacement peaks are (1,0), (2,0) and (4,0). All three imply velocity (1,0).

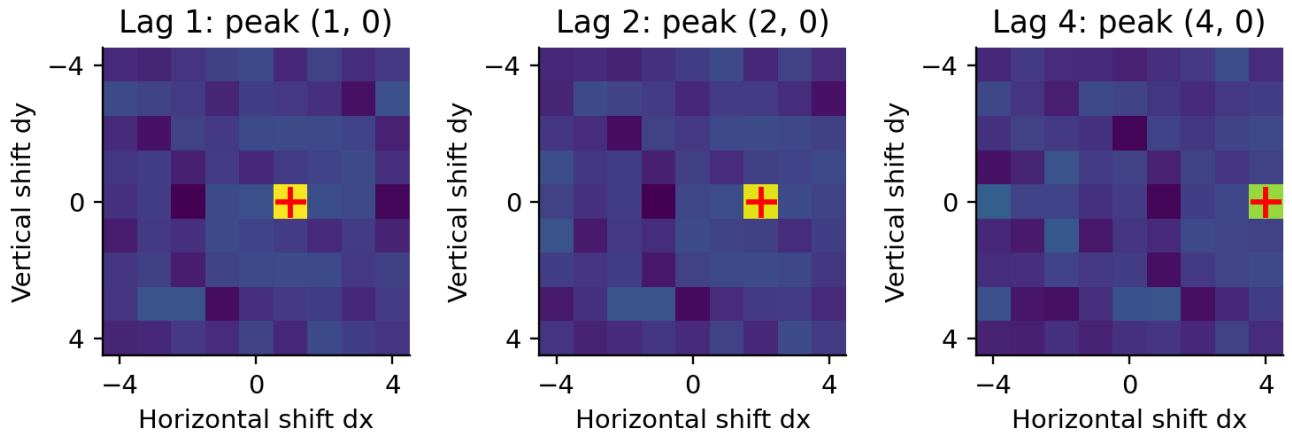


Figure 4. Actual fixed-bank outputs on a seeded synthetic translating texture, at frame 8 and site 44. Red crosses mark the observed maxima. These are input demonstrations, not trained-model results.

Why several lags? A short interval preserves correspondence during fast changes; a longer interval accumulates more displacement for slow movement. Their common displacement range gives different component-wise speed ranges: [-4,4], [-2,2] and [-1,1] pixels per frame. Longer intervals can also lose shared structure as dots disappear or enter the crop.

Startup arithmetic. At t=1, the lag-4 branch compares frame 0 with frame 1 but still divides its estimate by 4. At t=0 all three compare frame 0 with itself. The energy factor has no separate startup mask. These rules are part of the implemented boundary handling and matter when interpreting the first few frames.

The expectation can fall between integer candidates, yielding subpixel values. An equal mixture of (-3,0) and (+3,0) averages to zero; retaining p preserves their distinction from a population concentrated on (0,0).

9. Use motion to predict pixels

The estimate available on the previous frame predicts the current observation.

$$\mathbf{f}_{t,i} = \frac{1}{3} \sum_{\ell \in \{1,2,4\}} \widehat{\mathbf{v}}_{t,\ell,i}$$

f is the compact flow at site i: the simple average of the three velocity estimates. It has two components at each of 100 sites, giving a 2 by 10 by 10 field. The code bilinearly interpolates this field to 2 by 100 by 100. Interpolation forms weighted averages of neighboring site values. It retains pixel-per-frame units; it does not multiply velocities by the grid-size ratio.

$$\hat{I}_t(x,y) = \text{sample}\left(I_{t-1},\ x - f^x_{t-1}(x,y),\ y - f^y_{t-1}(x,y)\right)$$

$\hat{I}_t$ is the predicted current image. The superscripts x and y select the horizontal and vertical flow components. The sample operation reads the previous image, interpolating when its coordinates fall between pixels. The flow is from t-1, so this is a one-step continuation prediction: it asks whether the recently estimated motion explains the next image.

Why the minus sign?

If structure moves two pixels right, the value arriving at current position x came from old position x-2. The code visits each output pixel and asks where to read its source. This is called **backward sampling**, even though the supplied vector estimates forward motion. A check with a bright pixel at (40,40) and flow (2,-1) places the output peak at (42,39).

$$g_x = 2\frac{x - f^x}{99} - 1, \qquad g_y = 2\frac{y - f^y}{99} - 1$$

g_x and g_y are the normalized source coordinates passed to grid_sample: -1 is the first pixel center and +1 the last. Here align_corners=True defines that mapping. The separate flow upsampling uses align_corners=False. Out-of-image sampling uses the nearest border value. The image sampling itself is bilinear [3].

For example, at horizontal coordinate 10.25 the sampler combines columns 10 and 11 with weights 0.75 and 0.25. Applying the analogous rule vertically uses four neighboring pixels. This smoothly varying operation lets a pixel error send gradients back into the motion estimate; differentiable sampling is also the central enabling operation in spatial transformer networks [4].

10. Measure prediction error

The model asks how surprising the next image is relative to the available local signal.

$$e_{t,i} = \frac{1}{100} \sum_{(x,y) \in \Omega_i} [I_t(x,y) - 0.5]^2$$

Ω_i is the nonoverlapping 10 by 10 image block assigned to site i. Its 100 pixels are different from the larger 24 by 24 motion crop. e measures mean squared deviation from the fixed gray reference 0.5. It retains both brightness offsets and variations inside the block.

$$m_{t,i} = \frac{1}{100} \sum_{(x,y) \in \Omega_i} [\hat{I}_t(x,y) - I_t(x,y)]^2$$

$$\varepsilon_{t,i} = \frac{m_{t,i}}{\max((e_{t-1,i} + e_{t,i})/2, 10^{-4})}$$

m is mean squared pixel-prediction error. ε is that error divided by the average fixed-gray energy of the two frames, with a floor of 10^{-4}. The ratio makes a given error more substantial when the local signal is weak. If both frame energies are 0.04 and m=0.004, ε=0.1. The scalar ε is supplied to the learned visual features at that site. On frame zero, it is set to zero.

$$\omega_{t,i} = \frac{\sqrt{e_{t-1,i} e_{t,i}}}{\sqrt{e_{t-1,i} e_{t,i}} + 10^{-4}}, \qquad L_{\text{pixel}} = \text{mean}_{t \geq 1, i, \text{batch}} \omega_{t,i} \varepsilon_{t,i}$$

ω is another energy-based weight, this time for the pixel objective. It suppresses contributions from blocks with almost no fixed-gray energy in either frame. The mean averages weighted errors over noninitial frames, image sites and examples. In the preceding numerical example, ω is about 0.9975.

$$L = L_{\text{task}} + 0.05 L_{\text{pixel}}$$

L_{task} is the suite response loss: terminal cross-entropy, with the established value weighting for VDA. The factor 0.05 sets the auxiliary pixel objective's strength. This gives the refinement and temperature a reason to produce useful continuation predictions. Task gradients additionally teach which motion evidence matters for the response. A change, occlusion, dot replacement or new image can all increase the prediction error; its interpretation is learned in context.

11. Remember the recent population

An adaptive trace supports a local comparison on every frame.

Besides predicting the next pixels, the model compares current motion evidence with a running trace. Let $h_{t-1,\ell,i,k}$ be the value stored just before processing frame t. The trace has one entry for each site, lag and candidate displacement. At the beginning of every trial, every entry is 1/81.

$$\delta_{t,\ell,i} = \sum_{k=1}^{81} |p_{t,\ell,i,k} - h_{t-1,\ell,i,k}|$$

$$h_{t,\ell,i,k} = h_{t-1,\ell,i,k} + \eta r_{t,\ell,i}(p_{t,\ell,i,k} - h_{t-1,\ell,i,k})$$

δ is a summed absolute difference, also called L1 distance. Two normalized populations have a distance between 0 and 2. η is a learned update fraction obtained by applying sigmoid to one scalar parameter; sigmoid maps any real number into (0,1). η starts at 0.1. Reliability r slows trace updates when prepared signal is weak.

The order matters: compute δ from the **old** trace, use it in this frame's features, then update the trace. If ηr=0.1, the new trace is 90% old trace plus 10% current population. If ηr=0, it remains unchanged.

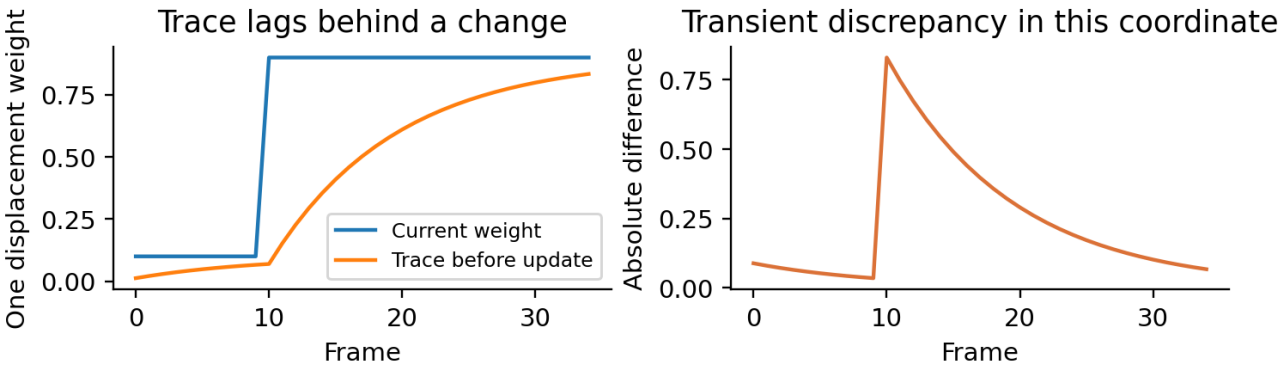


Figure 5. Teaching simulation of one population coordinate, η=0.1 and r=1. The full model adds absolute differences over all 81 coordinates.

After a stable motion changes, the current population moves toward different displacement candidates before the trace has caught up. That difference supplies a transient change signal. Over repeated observations the trace adapts, reducing the discrepancy. With a fixed effective update fraction 0.1, the influence of an old value decays by 0.9 per observation; its half-life is about 6.58 observations.

This trace and the next page's explicit time windows supply different descriptions. The trace has an exponentially fading history. The windows compare two finite, separated periods, preserving a defined earlier baseline for longer changes.

12. Compare two periods locally

The current population is pooled over a recent window and compared with an earlier reference window.

$$\mathcal{W}(t; w, g) = \{\max(0, t + 1 - g - w), \ldots, t - g\}$$

W names the included frame indices. w is the requested window width and g the count of most-recent frames excluded. A window is empty when its ending index is below zero. The recent window uses w=4, g=0; the reference uses w=8, g=8. Thus, once enough frames exist, recent covers t-3 through t and reference covers t-15 through t-8.

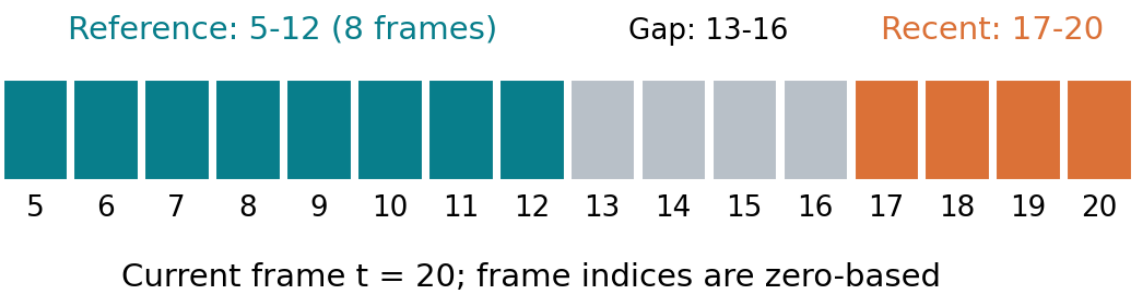


Figure 6. At t=20, reference frames 5-12 and recent frames 17-20 are separated by four unused frames, 13-16. The parameter gap=8 counts excluded recent frames, including the four-frame recent window.

$$\bar{p}_{t,\ell,i}^{(w,g)} = \frac{\sum_{s \in \mathcal{W}} r_{s,\ell,i} p_{s,\ell,i}}{\max(\sum_{s \in \mathcal{W}} r_{s,\ell,i}, 10^{-6})}$$

$$a_{t,\ell,i}^{(w,g)} = \frac{1}{w} \sum_{s \in \mathcal{W}} r_{s,\ell,i}$$

Here s is a frame inside the window. The vector p-bar is a reliability-weighted average of its populations. The scalar a is the total reliability divided by the requested width. It records both signal strength and how much of the window is available. A full window of perfectly reliable observations gives a=1. A half-filled window with the same reliability gives a=0.5.

Name the earlier average R and the recent average O. These letters mean reference and observation. Their joint weight is γ = square root of ($a_{reference}$ times a_{recent}). γ is zero when the reference has no observations. The first reference observation appears at t=8; it is frame zero. Full reference support appears at t=15. Therefore a seven-frame trial, t=0 through 6, has zero contribution from this explicit comparison branch.

Every quantity uses present or earlier observations. The windows run on every timestep with the same widths and gaps. They receive no cue times or change-event annotations. Cumulative sums implement these weighted boxcar averages efficiently.

13. What the comparison stores

Preserve the old pattern, the new pattern, and their differences at each location.

$$\gamma = \sqrt{a_R a_O}, \quad D_k = |O_k - R_k|, \quad d_1 = \sum_k D_k$$

$$d_{\cos} = 1 - \frac{\sum_k R_k O_k}{\max(|R|_2, 10^{-8})\max(|O|_2, 10^{-8})}$$

The site, time and lag indices are suppressed in these equations so the arithmetic is easier to read; all quantities still belong to one site and one lag. D is the 81-entry absolute difference. d_1 is its sum. The L2 norm $||R||_2$ is square root of the sum of squared entries. The cosine compares the directions of the two vectors; subtracting it from one gives a shape difference.

Stored component across 3 lags	Count	Purpose
10γR	243	Earlier local motion alternatives.
10γO	243	Recent local motion alternatives.
10γD	243	Which alternatives changed, coordinate by coordinate.
γd1	3	Overall absolute difference at each lag.
γdcos	3	Population-shape difference at each lag.
Total per site	735	An explicit local comparison vector.

The factor 10 is a fixed feature scale used for the population-valued components. It changes their magnitude before the learned readout; it is an engineering scale in this implementation. The scalar distances do not receive that factor. The same construction is repeated at all 100 image sites.

Why compare before pooling locations?

Suppose a left site initially represents rightward motion and a right site initially represents leftward motion. They then swap. Averaging over locations first gives the same half-rightward, half-leftward population before and after. Comparing each location first exposes a change at both locations. Spatial attention can then select the comparison at the relevant site.

If both window populations are uniform, their difference is zero. If one is concentrated entirely on rightward motion and the other entirely on upward motion, d_1=2 and d_{cos}=1. These are teaching extremes; actual softmax outputs usually distribute weight over multiple alternatives.

14. How motion reaches the answer

The motion pathway feeds both fused visual tokens and explicit local comparisons, selected by the same attention weights.

Input to each visual token	Number of values
Learned appearance responses + raw RGB/RMS statistics	32 + 4
Motion populations scaled by 10r	3 times 81 = 243
Energy reliabilities r	3
Differences δ from the adaptive trace	3
Expected horizontal/vertical velocities at 3 lags	6
Normalized pixel-prediction error ε	1
Concatenated input / learned output	292 / 64

$$V_{t,i} = W_{\text{fusion}}\phi_{t,i} + b_{\text{fusion}} + e_i$$

The Greek letter ϕ (phi) names the concatenated 292-entry feature list. W_{fusion} is a learned 64 by 292 matrix; b is its learned 64-entry bias. e_i is a learned position vector identifying site i. V is the resulting 64-entry visual token. Motion and appearance can influence the same token coordinates.

The recurrent attention module scores 100 site keys using one query derived from the previous global CLS memory. A softmax over sites gives weights $\alpha_{t,i}$, which add to one. Each key depends on its current visual token and previous local spatial memory. Consequently, remembered information changes which current site is selected.

$$b_t = \sum_{i=1}^{100} \alpha_{t,i}\, c_{t,i}, \qquad m_t = 0.8 m_{t-1} + 0.2 b_t$$

$c_{t,i}$ is the 735-entry local comparison from page 14. b_t is its attention-weighted spatial average. m_t is the 735-entry comparison memory, initially zero. The same α weights select learned visual values for the CLS update and modulate spatial memory writes. This is where the motion evidence participates in shared spatial selection.

At the final frame, the 64-entry CLS memory and the 735-entry comparison memory form a 799-entry vector. A learned linear layer and GELU turn it into 64 response features; the selected task head maps those features to answer logits. This separation allows the model to preserve detailed local motion comparisons alongside the compressed fused representation.

15. A complete numerical walkthrough

A synthetic texture moves right by exactly one pixel per frame; the fixed bank is executed unchanged.

The input is a seeded 100 by 100 random grayscale texture, with values between 0.2 and 0.8. Each new frame translates the texture one pixel right. At frame 8, inspect site 44: row 4, column 4. This interior crop stays away from the image wrapping boundary. No trained checkpoint is used; refinement is initially zero and temperature is 0.04.

Lag	Largest-score shift	Score C	r	Mean velocity (x,y)
1	(1, 0)	0.98631	0.99540	(1.0000, 0.0000)
2	(2, 0)	0.94733	0.99548	(1.0000, 0.0000)
4	(4, 0)	0.81390	0.99562	(1.0000, 0.0000)

The peak displacement increases with elapsed time. Dividing by lag returns approximately the same one-pixel-per-frame velocity. The longer-lag score can be lower because the feature has moved farther under the fixed Hann window, and crop boundaries change the overlapping evidence.

Read the computation in sequence

1. At each lag, retrieve the earlier crop and the current crop. Subtract their respective means and apply the same Hann window. **2.** Calculate 81 product sums and divide by the full-crop energy normalization. **3.** Convert the 81 scores into normalized weights with softmax. **4.** Weight the candidate shift coordinates by those values and divide by the lag. **5.** Average the three lag velocities to form the flow used for the next prediction.

6. Compare the current population with the old adaptive trace. **7.** Form the earlier and recent window populations where history exists. **8.** Supply motion features to fusion and supply local comparisons to shared attention. All these operations retain the location until the attention-weighted selection.

Checks executed for this document

The actual CPU motion bank recovers maxima (1,0), (2,0), (4,0). Direct product sums match the FFT scores. A separate impulse-image check verifies the sign of backward sampling. A flat midpoint-gray movie gives zero score and zero reliability. The reference window is empty at t=7, has reliability mass divided by width equal to 1/8 at t=8, and reaches full support at t=15 when every frame is reliable.

All numerical outputs and source hashes are preserved in output/pdf/optic_flow_pathway_verification.json. Synthetic demonstrations illustrate computation, rather than task accuracy.

16. The scientific ideas behind it

Local motion evidence, distributed alternatives and normalization provide the theoretical starting points.

Motion as orientation in space and time [5]

Imagine a horizontal row of image pixels stacked over successive frames. A dot moving right traces a diagonal line in this space-time picture. The slope relates position change to elapsed time. Adelson and Bergen constructed motion-energy models with filters oriented in space-time: paired filter outputs are squared and summed, then opposite directions are compared. Their construction explains how direction selectivity can arise from image signals without assigning an identity to each dot.

Our connection is the underlying use of **spatial relationships across time**. The implemented bank measures alignment at explicit displacements and lags. Its central operation is the product-sum correlation derived above. The energy-model paper supplies a theoretical account of why spatiotemporal structure supports motion sensing.

A population can retain ambiguity [6]

Simoncelli and Heeger modeled responses in visual areas V1 and MT through weighted inputs, rectification and normalization. Their model represents local velocity using a distribution of neuronal responses. A population permits multiple directions and speeds to be expressed together; the pattern across units carries information that a single winning unit would discard.

In our implementation, the 81 coordinates have explicitly assigned displacements. Keeping their joint pattern is useful when the image supports multiple matches. The expected velocity is one summary of that pattern. The larger population remains available for detecting changes even when opposing alternatives cancel in the mean.

Normalization separates two questions

The crop-energy denominator asks how well the prepared structures align relative to their scale. The reliability factor separately carries their signal strength. This is an engineering decomposition motivated by a common computational need: preserve which alternative fits while controlling the effect of input magnitude. The code states exactly how the two quantities are constructed.

Learning uses both prediction and the task

The pixel objective rewards a flow that predicts continuing image structure. The response objective rewards representations useful for the behavioral judgment. Together they can shape refinement and selection. The finite crop, candidate grid, chosen lags and temporal windows specify the problems that learning is asked to solve within this architecture.

17. Fixed rules and learned quantities

The method combines an explicit correspondence calculation with trainable interpretation.

Component	Fixed or learned?	Role
24-pixel crop; stride 10; Hann window	Fixed	Defines local image support.
Lags 1, 2, 4; shifts -4 to +4	Fixed	Defines the measured temporal/spatial alternatives.
FFT correlation and energy normalization	Fixed arithmetic	Produces image-derived match scores.
1 to 4 to 1 convolutional refinement	Learned: 77 parameters	Adjusts local structure in displacement-score space.
Three temperatures	Learned, bounded	Controls sharpness separately at each lag.
Adaptive-trace update fraction	Learned scalar	Controls rate of adaptation.
Window widths/gaps and comparison decay 0.8	Fixed	Specifies explicit temporal comparisons.
Fusion, recurrent attention and response layers	Learned	Uses motion and appearance to solve the task.

The first refinement convolution has 4 times 1 times 3 times 3 weights plus four biases, giving 40 parameters. The second has 1 times 4 times 3 times 3 weights plus one bias, giving 37. Together with three temperatures and one trace-rate parameter, these add 81 learned parameters around the fixed motion computations. Fusion and recurrent processing contain additional learned parameters.

Why these pieces fit together

Raw differences between images mix motion with changes in appearance. Correlation instead evaluates candidate correspondences. The population preserves alternatives; the mean provides a compact displacement. Prediction tests temporal continuation; traces and windows detect changes in evidence. Shared attention identifies which local evidence should influence memory. The task head translates the selected memory into its response vocabulary.

This arrangement builds correspondence and temporal comparison into the representation. Those choices explain the pathway's usefulness and define concrete things to test: shift range versus stimulus speed, crop size versus object size, lag versus dot lifespan, and window support versus trial duration. Each test changes a specific part of the computational chain.

18. Symbol and units reference

The same symbols keep the same roles across this document.

Symbol	Meaning	Shape / units
X; I	RGB input; channel-mean grayscale	3 x 100 x 100; 100 x 100 per frame
t; ℓ; i	Frame index; lag; spatial site	Integer; frame steps; 0-99
x,y; a,b	Image coordinates; crop coordinates	Pixels; integers 0-23
Q; P	Extracted centered crop; prepared crop	24 x 24 each
w; N; C	Hann weight; raw correlation; normalized score	Scalar at each coordinate or shift
E; z; r	Prepared crop energy; joint energy; reliability	Scalar; scalar; fraction in [0,1)
p; τ	Displacement population; temperature	81 weights per site/lag; positive scalar
d; v; f	Displacement; lag velocity; mean-lag flow	2D: pixels; pixels/frame; pixels/frame
I-hat; ε; ω	Predicted image; normalized error; loss weight	Image; scalar/site; scalar/site
h; η; δ	Population trace; trace update rate; L1 distance	81 values; fraction; scalar/site/lag
R; O; γ	Reference; recent population; joint support	81 values each; scalar/site/lag
c; α; m	Local comparison; attention; comparison memory	735/site; fraction/site; 735 global
ϕ; V	Concatenated features; fused token	292/site; 64/site

E denotes a sum of prepared squared values on page 6, whereas e on page 11 denotes an average squared deviation from fixed gray over a smaller block. These two energy measurements use different spatial supports and centering rules. The position vector e_i on page 15 is explicitly labeled there; its 64-entry shape distinguishes it from the scalar image energy $e_{t,i}$.

Reading operators: |x| is absolute value; ||v||2 is square root of the sum of squared entries; exp is the exponential; max(a,b) chooses the larger value; a subscript selects an index; a superscript can label a component or window. The arithmetic meaning is specified immediately below each equation.

19. References and code provenance

Primary sources explain the ideas; the inspected source code defines the implemented equations.

[1] Dosovitskiy, A. et al. (2015). FlowNet: Learning Optical Flow with Convolutional Networks. ICCV.

https://arxiv.org/abs/1504.06852
Connection: correlation volumes expose matching evidence to learned processing.

[2] PyTorch. torch.fft.irfft2 documentation.

https://docs.pytorch.org/docs/2.14/generated/torch.fft.irfft2.html
API reference for the real inverse transform. Local numerical checks verify the running implementation.

[3] PyTorch. torch.nn.functional.grid_sample documentation.

https://docs.pytorch.org/docs/2.14/generated/torch.nn.functional.grid_sample.html
API reference for source coordinates, bilinear sampling, border handling and align_corners.

[4] Jaderberg, M., Simonyan, K., Zisserman, A. and Kavukcuoglu, K. (2015). Spatial Transformer Networks. NeurIPS.

https://arxiv.org/abs/1506.02025
Connection: differentiable sampling allows image errors to influence spatial transformations.

[5] Adelson, E. H. and Bergen, J. R. (1985). Spatiotemporal energy models for the perception of motion. JOSA A 2(2), 284-299.

https://doi.org/10.1364/JOSAA.2.000284
Connection: direction-sensitive computations from image structure across space and time.

[6] Simoncelli, E. P. and Heeger, D. J. (1998). A model of neuronal responses in visual area MT. Vision Research 38(5), 743-761.

https://www.cns.nyu.edu/pub/eero/simoncelli96.pdf
Connection: distributed motion representation and normalization in a model of V1/MT.

Implementation record

Verified October 12, 2026 against continuation07/config.json source_sha256. The actual suite inherits the stabilized backbone through StabilizedAttentionSuite and StabilizedAttentionVDA. Local training uses PyTorch 1.13.1+cu117; the cited API pages are current documentation. The sign, normalization and startup claims were also checked directly on CPU with the local library.

Source entry points: StabilizedAttentionKrauzlis/model.py: LocalMotionBank (lines 14-36), warp (39-45), causal_average (48-60), population/comparison construction (96-114), prediction/trace/fusion (117-141), attention routing (155-171). MemoryGuidedSearch/train.py: update(), task loss plus 0.05 prediction loss. Full paths and SHA256 values are recorded in the companion verification JSON.